Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ РОБОТИ № 3

з дисципліни «Професійний Python»

на тему: «Ітератори, генератори та потокова обробка даних»

Виконала:
студентка 3 курсу
групи ФеП-33
Наборська Дарія
Русланівна

Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

Тема роботи

Лабораторна робота № 3. Ітератори, генератори та потокова обробка даних.

Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

Номер індивідуального варіанта

Варіант 9. Потокова обробка спортивних результатів.

Постановка задачі

Розробити потоковий конвеєр обробки великого набору спортивних результатів. Input: CSV із полями athlete, sport, result, category, timestamp. Реалізувати:

- streaming validation спортивних результатів;
- фільтрацію за видом спорту та категорією;
- визначення найкращого результату;
- streaming average result;
- формування рейтингу результатів;
- pairwise аналіз зміни послідовних результатів;
- batching;
- отримання перших N результатів за заданою умовою.

Конвеєр має містити джерело даних, generators для читання, очищення, parsing, filtering і transformation, aggregation stage, generator expression, щонайменше три функції з yield, yield from, щонайменше три засоби itertools, islice, streaming і batch processing, обробку не менше 100 000 records, порівняння eager і lazy variants, оцінку memory consumption та аналіз переваг і недоліків lazy evaluation.

Завдання підвищеної складності

Для максимальної оцінки потрібно щонайменше три додаткові можливості. Реалізовано:

- **власний iterator class** — ResultsIterator та iterable QualifiedResults;
- **recursive generator із yield from** — flatten;
- **infinite generator + islice** — lap_numbers на основі itertools.count;
- **itertools.chain, accumulate, pairwise, groupby** — читання кількох files, running average, зміна результатів, найкращий результат за видом спорту;
- **dynamic batching** — dynamic_batches з обмеженням суми ваг;

- **pipeline із декількох files** — read_many([results_100000.csv, sample.csv]);
- **streaming export у новий file** — export_records записує МСМК-результати в output/msmk_results.csv;
- **підрахунок invalid records** — StreamStats з причинами помилок;
- **logging pipeline stages** — StageLog рахує записи на кожному stage;
- **measurement time-to-first-result, lazy search із early termination, memory list vs generator** — модуль experiments.py.

Репозиторій: https://github.com/dashanaborska/python_lab, каталог Lab_03 (звіт — у Lab_03/report/).

Опис input dataset

Набір даних — CSV-файл у кодуванні UTF-8 з заголовком:

```csv
athlete,sport,result,category,timestamp
```

Поле	Тип після parsing	Опис
athlete	str	ім'я та прізвище спортсмена
sport	str	вид спорту (Плавання, Бокс, Легка атлетика, Велоспорт)
result	float	результат у балах, допустимий діапазон 0–100
category	str	МСМК, МС, КМС, I/II/III розряд
timestamp	datetime	час результату у форматі ISO 8601

Великі набори генерує модуль generator.py (sport-results-generate N): 100 імен спортсменів, кожен закріплений за одним видом спорту, результат з нормальним розподілом (середнє 70, σ = 10), timestamp зростає щохвилини. Приблизно 2 % рядків навмисно пошкоджені: нечисловий результат, результат > 100, невідома категорія або пропущені поля. Файл data/sample.csv містить 12 рядків, з них 4 некоректні та один з зайвими пробілами в імені. Для експериментів створено файли на 10 000, 100 000, 500 000 записів і потік на 1 000 000 записів.

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03> sport-results-generate 100000
  Створено data\generated\results_100000.csv (100,000 записів, 7.68 MB)
```

Рисунок 1 — Генерація тестового набору на 100 000 записів


```csv
athlete,sport,result,category,timestamp
```

```
Олена Ковальчук,Плавання,87.5,КМС,2026-03-01T09:00:00
Максим Бондар,Легка атлетика,92.3,МС,2026-03-01T09:05:00
Ірина Мельник,Плавання,78.4,I розряд,2026-03-01T09:10:00
Андрій Шевченко,Бокс,81.0,КМС,2026-03-01T09:15:00
Софія Ткачук,Легка атлетика,95.1,МСМК,2026-03-01T09:20:00
Тарас Новак,Бокс,n/a,МС,2026-03-01T09:25:00
Марія Бойко,Плавання,90.2,МС,2026-03-01T09:30:00
  дмитро   савчук ,Велоспорт,73.8,II розряд,2026-03-01T09:35:00
Юлія Кравець,Велоспорт,124.0,КМС,2026-03-01T09:40:00
Богдан Лисенко,Легка атлетика,69.5,майстер,2026-03-01T09:45:00
Катерина Гнатюк,Бокс,86.4,КМС,2026-03-01T09:50:00
Олег Петренко,Плавання,91.7,МС,
```

Структура проєкту

None

```
python_lab/
├── README.md
├── Lab_01/  Lab_02/
└── Lab_03/
    ├── .gitignore
    ├── pyproject.toml
    ├── README.md
    ├── data/
    │   ├── sample.csv
    │   └── generated/        (генерується, у .gitignore)
    ├── report/
    │   └── Lab_03_Naborska.pdf
    ├── src/
    │   └── sport_results/
    │       ├── __init__.py
    │       ├── analytics.py      aggregation, rating, pairwise, groupby
    │       ├── batching.py       batched, dynamic_batches
    │       ├── config.py         константи
    │       ├── experiments.py    eager vs lazy, memory, early termination
    │       ├── export.py         streaming export
    │       ├── filters.py        filtering, islice, find_first
    │       ├── generator.py      генерація dataset
    │       ├── iterators.py      власний iterator, infinite і recursive
generators
    │       ├── main.py           точка входу
    │       ├── models.py         SportResult, StreamStats
    │       ├── parsers.py        clean, validate, parse
    │       ├── pipeline.py       lazy pipeline, StageLog
    │       ├── readers.py        streaming readers
    │       └── transform.py      transformation
    └── tests/
        └── test_stream.py
```

Python

```
@dataclass(frozen=True)
class SportResult:
    athlete: str
    sport: str
    result: float
    category: str
    timestamp: datetime
```

```
@dataclass
class StreamStats:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    errors: dict[str, int] = field(default_factory=dict)

    def add_error(self, reason: str) -> None:
        self.invalid += 1
        self.errors[reason] = self.errors.get(reason, 0) + 1
```

Пояснення iterable та iterator

Iterable — об'єкт, з якого можна отримати iterator викликом iter() (має метод __iter__): list, str, file object, QualifiedResults. Iterator — об'єкт, що зберігає поточний стан обходу, має __next__() і після завершення піднімає StopIteration; його метод __iter__() повертає сам iterator, тому iter(iterator) is iterator. Цикл for спочатку викликає iter(), а потім next() до StopIteration. Iterable можна обходити багато разів (щоразу створюється новий iterator), iterator — лише один раз.

Python

```
values = [10, 20, 30]
iterator = iter(values)
next(iterator), next(iterator), next(iterator)
iter(iterator) is iterator
```

Код власного iterator

ResultsIterator реалізує протокол __iter__/__next__ і повертає лише результати не нижче мінімуму. QualifiedResults — окремий iterable, який для кожного проходу створює новий iterator, тому його можна обходити повторно (видно на рисунку 2).

Python

```
class ResultsIterator:
    def __init__(self, records: list[SportResult], minimum: float) -> None:
        self._records = records
        self._minimum = minimum
        self._index = 0

    def __iter__(self) -> "ResultsIterator":
        return self

    def __next__(self) -> SportResult:
        while self._index < len(self._records):
            record = self._records[self._index]
            self._index += 1
            if record.result >= self._minimum:
                return record
        raise StopIteration


class QualifiedResults:
```

```
    def __init__(self, records: list[SportResult], minimum: float) -> None:
        self.records = records
        self.minimum = minimum

    def __iter__(self) -> ResultsIterator:
        return ResultsIterator(self.records, self.minimum)
```

Приклад generator function

Generator function містить yield і при виклику повертає generator object, не виконуючи тіла. Виконання починається з першого next() і зупиняється на кожному yield, зберігаючи локальні змінні. Читання файлу рядок за рядком:

```Python
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8", newline="") as file:
        for line in file:
            yield line
```

Приклад yield

Функції з yield у проєкті: read_lines, read_rows, clean_rows, validate, parse_rows, filter_records, normalize, batched, dynamic_batches, running_average, result_changes, batch_averages, StageLog._count, lap_numbers, flatten. Наприклад, перетворення рядків у SportResult:

```Python
def parse_rows(rows: Iterable[dict[str, str]]) -> Iterator[SportResult]:
    for row in rows:
        yield SportResult(
            athlete=row["athlete"],
            sport=row["sport"],
            result=float(row["result"]),
            category=row["category"],
            timestamp=datetime.fromisoformat(row["timestamp"]),
        )
```

Приклад yield from

```Python
def read_rows(path: Path) -> Iterator[dict[str, str]]:
    with path.open("r", encoding="utf-8", newline="") as file:
        yield from csv.DictReader(file)


def read_many(paths: Iterable[Path]) -> Iterator[dict[str, str]]:
    yield from chain.from_iterable(read_rows(path) for path in paths)


def lap_numbers(start: int = 1) -> Iterator[int]:
    yield from count(start)
```

```
def flatten(values: Iterable[Any]) -> Iterator[Any]:
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value
```

yield from делегує генерацію іншому iterable: read_rows віддає рядки csv.DictReader, read_many — рядки з усіх файлів через chain.from_iterable, flatten рекурсивно розгортає вкладені списки, lap_numbers — нескінченний itertools.count, обмежений через islice.

Generator expression

Python

```
def filter_by(
    records: Iterable[SportResult],
    predicate: Predicate,
) -> Iterator[SportResult]:
    return (record for record in records if predicate(record))


def read_many(paths: Iterable[Path]) -> Iterator[dict[str, str]]:
    yield from chain.from_iterable(read_rows(path) for path in paths)
```

Generator expression (record for record in records if predicate(record)) не створює проміжного списку: кожен елемент обчислюється при запиті. Також generator expressions використано в read_many, running_average (record.result for record in records) і batch_averages.

Використані засоби itertools

Засіб	Де використано	Призначення
islice	batched, take_first, main	обмеження потоку, перші N записів, batches
chain.from_iterable	read_many	об'єднання кількох файлів в один потік
count	lap_numbers	нескінченний генератор номерів
accumulate	running_average	накопичувальна сума для running average
pairwise	result_changes	пари послідовних результатів
groupby	best_by_sport	групування за видом спорту після sorted

Схема pipeline

```
data/generated/results_100000.csv + data/sample.csv
        ↓
read_many()  — chain, yield from csv.DictReader      (source / streaming
reader)
        ↓
clean_rows() — strip пробілів                         (cleaning)
        ↓
validate()   — check_row, StreamStats                 (validation)
        ↓
parse_rows() — dict → SportResult                     (parsing)
        ↓
filter_records(sport, category)                       (filtering)
        ↓
normalize()  — ім'я Title Case, round(result, 1)      (transformation)
        ↓
batched() / streaming_average / build_rating / export (batching /
aggregation / sink)
        ↓
Result
```

```python
class StageLog:
    def __init__(self) -> None:
        self.counts: dict[str, int] = {}

    def track(self, name: str, items: Iterable[T]) -> Iterator[T]:
        self.counts[name] = 0
        return self._count(name, items)

    def _count(self, name: str, items: Iterable[T]) -> Iterator[T]:
        for item in items:
            self.counts[name] += 1
            yield item


def build_pipeline(
    paths: Iterable[Path],
    stats: StreamStats,
    sport: str | None = None,
    category: str | None = None,
    log: StageLog | None = None,
) -> Iterator[SportResult]:
    stage_log = log or StageLog()
    rows = stage_log.track("read", read_many(paths))
    cleaned = stage_log.track("clean", clean_rows(rows))
    valid = stage_log.track("validate", validate(cleaned, stats))
    parsed = stage_log.track("parse", parse_rows(valid))
    selected = stage_log.track(
        "filter", filter_records(parsed, sport, category)
    )
    return stage_log.track("transform", normalize(selected))
```

Кожен stage обгорнуто StageLog.track, що лениво підраховує записи; кожен record проходить усі stages лише тоді, коли його запитує наступний stage.

Код streaming reader

```python
import csv
from collections.abc import Iterable, Iterator
from itertools import chain
from pathlib import Path


def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8", newline="") as file:
        for line in file:
            yield line


def read_rows(path: Path) -> Iterator[dict[str, str]]:
    with path.open("r", encoding="utf-8", newline="") as file:
        yield from csv.DictReader(file)


def read_many(paths: Iterable[Path]) -> Iterator[dict[str, str]]:
    yield from chain.from_iterable(read_rows(path) for path in paths)


def read_all_rows(path: Path) -> list[dict[str, str]]:
    with path.open("r", encoding="utf-8", newline="") as file:
        return list(csv.DictReader(file))
```

read_lines і read_rows обробляють файл порядково: file object є iterable, тому в пам'яті одночасно знаходиться лише поточний рядок. read_all_rows — eager-варіант для порівняння.

Validation stage

```python
def clean_rows(rows: Iterable[dict[str, str]]) -> Iterator[dict[str, str]]:
    for row in rows:
        yield {
            key.strip(): (value or "").strip()
            for key, value in row.items()
            if key is not None
        }


def check_row(row: dict[str, str]) -> str | None:
    if any(not row.get(name) for name in FIELDS):
        return "missing field"
    try:
        result = float(row["result"])
    except ValueError:
        return "result is not a number"
    if not MIN_RESULT <= result <= MAX_RESULT:
        return "result out of range"
    if row["category"] not in CATEGORY_SET:
        return "unknown category"
    try:
        datetime.fromisoformat(row["timestamp"])
    except ValueError:
```

```
        return "bad timestamp"
    return None


def validate(
    rows: Iterable[dict[str, str]],
    stats: StreamStats,
) -> Iterator[dict[str, str]]:
    for row in rows:
        stats.total += 1
        reason = check_row(row)
        if reason is None:
            stats.valid += 1
            yield row
        else:
            stats.add_error(reason)
```

Некоректні записи не зупиняють потік: вони підраховуються в StreamStats за причиною, а далі передаються лише коректні.

Filtering stage

Python

```
from collections.abc import Callable, Iterable, Iterator
from itertools import islice

from sport_results.models import SportResult

Predicate = Callable[[SportResult], bool]


def filter_records(
    records: Iterable[SportResult],
    sport: str | None = None,
    category: str | None = None,
) -> Iterator[SportResult]:
    for record in records:
        if sport is not None and record.sport != sport:
            continue
        if category is not None and record.category != category:
            continue
        yield record


def filter_by(
    records: Iterable[SportResult],
    predicate: Predicate,
) -> Iterator[SportResult]:
    return (record for record in records if predicate(record))


def take_first(
    records: Iterable[SportResult],
    count: int,
    predicate: Predicate,
) -> list[SportResult]:
    return list(islice(filter_by(records, predicate), count))
```

```
def find_first(
    records: Iterable[SportResult],
    predicate: Predicate,
) -> SportResult | None:
    for record in records:
        if predicate(record):
            return record
    return None
```

Transformation stage

Python

```
from collections.abc import Iterable, Iterator
from dataclasses import replace

from sport_results.models import SportResult


def normalize(records: Iterable[SportResult]) -> Iterator[SportResult]:
    for record in records:
        yield replace(
            record,
            athlete=" ".join(record.athlete.split()).title(),
            result=round(record.result, 1),
        )


def to_row(record: SportResult) -> dict[str, str]:
    return {
        "athlete": record.athlete,
        "sport": record.sport,
        "result": f"{record.result:.1f}",
        "category": record.category,
        "timestamp": record.timestamp.isoformat(),
    }
```

Aggregation stage

Python

```
import heapq
from collections.abc import Iterable, Iterator
from itertools import accumulate, groupby, pairwise
from operator import attrgetter

from sport_results.models import SportResult


def best_result(records: Iterable[SportResult]) -> SportResult | None:
    return max(records, key=attrgetter("result"), default=None)


def streaming_average(records: Iterable[SportResult]) -> float:
    count = 0
    total = 0.0
    for record in records:
        count += 1
        total += record.result
```

```
    return total / count if count else 0.0


def running_average(records: Iterable[SportResult]) -> Iterator[float]:
    totals = accumulate(record.result for record in records)
    for number, total in enumerate(totals, start=1):
        yield total / number


def build_rating(
    records: Iterable[SportResult],
    top: int = 10,
) -> list[tuple[int, SportResult]]:
    leaders = heapq.nlargest(top, records, key=attrgetter("result"))
    return list(enumerate(leaders, start=1))


def result_changes(
    records: Iterable[SportResult],
) -> Iterator[tuple[SportResult, SportResult, float]]:
    for previous, current in pairwise(records):
        yield previous, current, round(current.result - previous.result, 2)


def best_by_sport(records: Iterable[SportResult]) -> dict[str, float]:
    ordered = sorted(records, key=attrgetter("sport"))
    return {
        sport: max(record.result for record in group)
        for sport, group in groupby(ordered, key=attrgetter("sport"))
    }


def batch_averages(batches: Iterable[list[SportResult]]) -> Iterator[float]:
    for batch in batches:
        yield sum(record.result for record in batch) / len(batch)
```

streaming_average рахує середнє за один прохід без збереження записів; build_rating використовує heapq.nlargest, який тримає в пам'яті лише top N елементів; result_changes через pairwise показує зміну послідовних результатів; best_by_sport використовує groupby (перед ним потрібне sorted, бо groupby групує лише сусідні елементи — тут sorted матеріалізує дані, що є свідомим компромісом).

Batching

Python

```
from collections.abc import Callable, Iterable, Iterator
from itertools import islice
from typing import TypeVar

T = TypeVar("T")


def batched(items: Iterable[T], size: int) -> Iterator[list[T]]:
    if size < 1:
        raise ValueError("Розмір batch має бути додатним.")
    iterator = iter(items)
```

```
    while batch := list(islice(iterator, size)):
        yield batch


def dynamic_batches(
    items: Iterable[T],
    limit: float,
    weight: Callable[[T], float],
) -> Iterator[list[T]]:
    batch: list[T] = []
    total = 0.0
    for item in items:
        item_weight = weight(item)
        if batch and total + item_weight > limit:
            yield batch
            batch, total = [], 0.0
        batch.append(item)
        total += item_weight
    if batch:
        yield batch
```

batched ділить потік на списки фіксованого розміру за допомогою islice; dynamic_batches формує batches за сумарною «вагою» записів.

Результат роботи програми

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03> python -m pip install -e .
  Attempting uninstall: sport-results
    Found existing installation: sport-results 0.3.0
    Uninstalling sport-results-0.3.0:
      Successfully uninstalled sport-results-0.3.0
Successfully installed sport-results-0.3.0
```

Рисунок 2 — Встановлення пакета у virtual environment

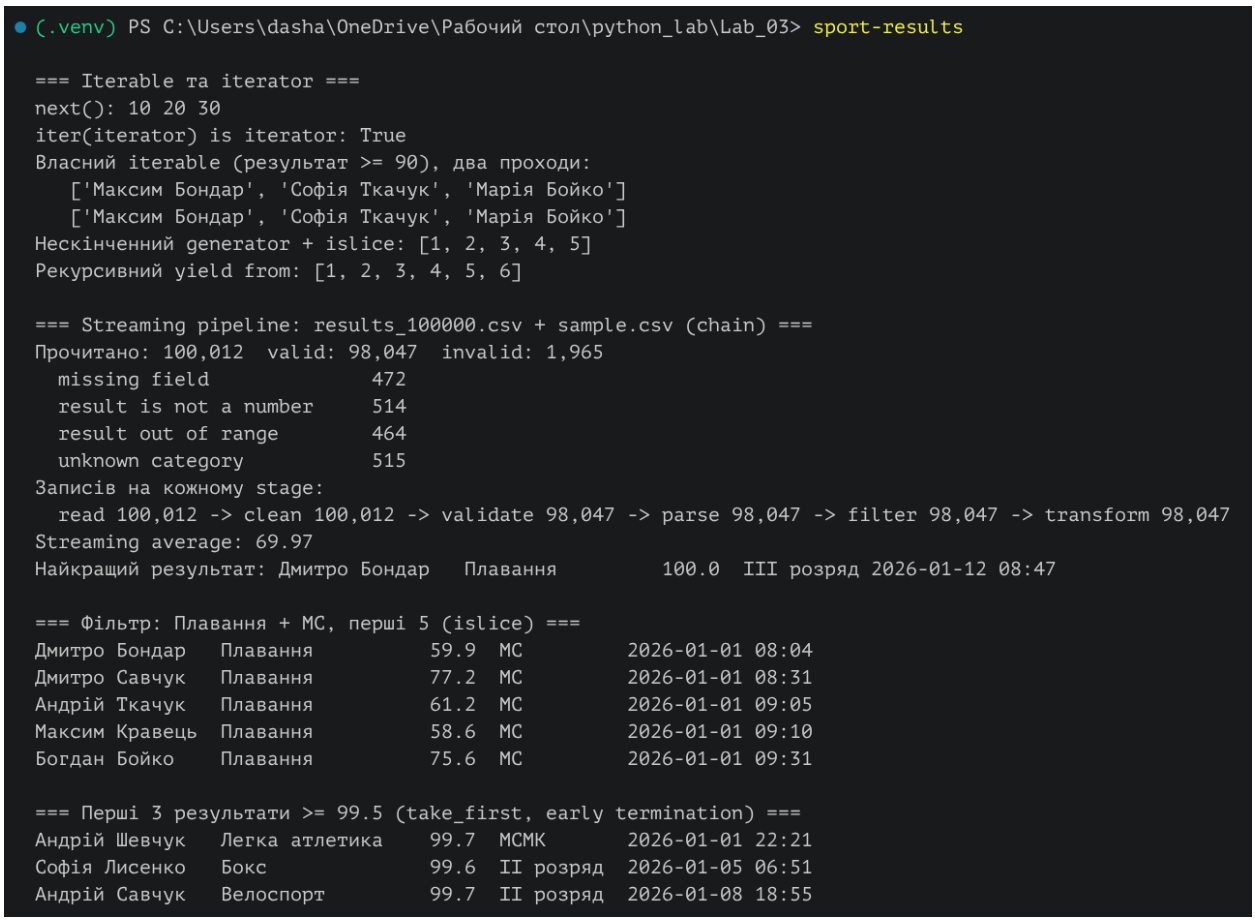


Рисунок 3 — Iterator, generators, streaming pipeline, validation, фільтрація, перші N результатів

```
=== Рейтинг топ-5 (heapq.nlargest по потоку) ===
1. Тарас Савчук    Бокс            100.0  КМС        2026-02-03 11:43
2. Тарас Шевчук    Бокс             99.9  I розряд   2026-02-24 20:26
3. Андрій Коваль   Бокс             99.8  II розряд  2026-01-14 04:28
4. Максим Новак    Бокс             99.8  МСМК       2026-01-14 09:56
5. Марія Шевчук    Бокс             99.8  I розряд   2026-02-12 21:58

=== Найкращий результат за видом спорту (groupby) ===
Бокс             100.0
Велоспорт         99.9
Легка атлетика    99.7
Плавання         100.0

=== Pairwise: зміна результатів спортсмена Олена Коваль ===
  74.2 ->   75.0  зміна  +0.80
  75.0 ->   88.0  зміна +13.00
  88.0 ->   58.1  зміна -29.90
  58.1 ->   71.5  зміна +13.40
  71.5 ->   74.1  зміна  +2.60

=== Running average (accumulate), перші 5 значень ===
[70.5, 69.15, 68.3, 68.35, 66.66]

=== Batch processing по 25 000 записів ===
batch 1: середнє 70.00
batch 2: середнє 69.82
batch 3: середнє 70.05
batch 4: середнє 70.01

=== Dynamic batching (сума результатів <= 500) ===
розміри batch: [5, 3]
Streaming average для sample.csv: 85.59
Найкращий у sample.csv: Софія Ткачук

=== Streaming export ===
Записано 16,274 рядків у output\msmk_results.csv
```

Рисунок 4 — Рейтинг, groupby, pairwise, accumulate, batching, streaming export

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03\.venv\Scripts\python.exe
rootdir: C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03
configfile: pyproject.toml
collecting ... collected 16 items

tests/test_stream.py::test_read_lines_is_lazy PASSED                     [  6%]
tests/test_stream.py::test_read_many_chains_files PASSED                 [ 12%]
tests/test_stream.py::test_validation_counts_invalid PASSED              [ 18%]
tests/test_stream.py::test_check_row[abc-result is not a number] PASSED  [ 25%]
tests/test_stream.py::test_check_row[101-result out of range] PASSED     [ 31%]
tests/test_stream.py::test_lazy_exception_on_consumption PASSED          [ 37%]
tests/test_stream.py::test_pipeline_normalizes_and_logs PASSED           [ 43%]
tests/test_stream.py::test_filter_records PASSED                         [ 50%]
tests/test_stream.py::test_take_first_and_find_first PASSED              [ 56%]
tests/test_stream.py::test_analytics PASSED                              [ 62%]
tests/test_stream.py::test_batching PASSED                               [ 68%]
tests/test_stream.py::test_custom_iterator PASSED                        [ 75%]
tests/test_stream.py::test_infinite_and_recursive_generators PASSED      [ 81%]
tests/test_stream.py::test_exhausted_generator PASSED                    [ 87%]
tests/test_stream.py::test_generate_and_export PASSED                    [ 93%]
tests/test_stream.py::test_experiment_helpers PASSED                     [100%]

============================== 16 passed in 0.37s ==============================
```

Рисунок 5 — Результат виконання тестів pytest

Eager implementation

```python
def eager_process(path: Path) -> tuple[int, float]:
    rows = read_all_rows(path)
```

```
    cleaned = list(clean_rows(rows))
    valid = [row for row in cleaned if check_row(row) is None]
    parsed = list(parse_rows(valid))
    selected = [record for record in parsed if record.sport == SPORT]
    total = sum(record.result for record in selected)
    return len(selected), total / len(selected)


def eager_first(path: Path) -> SportResult:
    rows = read_all_rows(path)
    parsed = list(parse_rows(row for row in rows if check_row(row) is None))
    return [record for record in parsed if record.sport == SPORT][0]
```

Eager-варіант на кожному кроці будує повний список: усі рядки файлу, очищені рядки, валідні рядки, об'єкти SportResult і відфільтровані записи.

Lazy implementation

Python

```
def lazy_process(path: Path) -> tuple[int, float]:
    count = 0
    total = 0.0
    for record in build_pipeline([path], StreamStats(), sport=SPORT):
        count += 1
        total += record.result
    return count, total / count


def lazy_first(path: Path) -> SportResult:
    return next(build_pipeline([path], StreamStats(), sport=SPORT))
```

Lazy-варіант проходить файл один раз через ланцюжок generators і зберігає лише лічильник та суму. Для коректного вимірювання pipeline повністю споживається в циклі for.

Python

```
def elapsed(function: Callable[..., Any], *args: Any) -> float:
    start = perf_counter()
    function(*args)
    return perf_counter() - start


def peak_memory(function: Callable[..., Any], *args: Any) -> float:
    tracemalloc.start()
    function(*args)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024 / 1024


def early_termination(count: int) -> tuple[float, float, int]:
    start = perf_counter()
    lazy_stats = StreamStats()
    lazy = find_first(
        parse_rows(validate(synthetic_rows(count), lazy_stats)), is_target
    )
    lazy_time = perf_counter() - start
```

```
    start = perf_counter()
    parsed = list(parse_rows(validate(synthetic_rows(count), StreamStats())))
    matches = [record for record in parsed if is_target(record)]
    eager = matches[0] if matches else None
    eager_time = perf_counter() - start
    assert lazy == eager
    return eager_time, lazy_time, lazy_stats.total
```

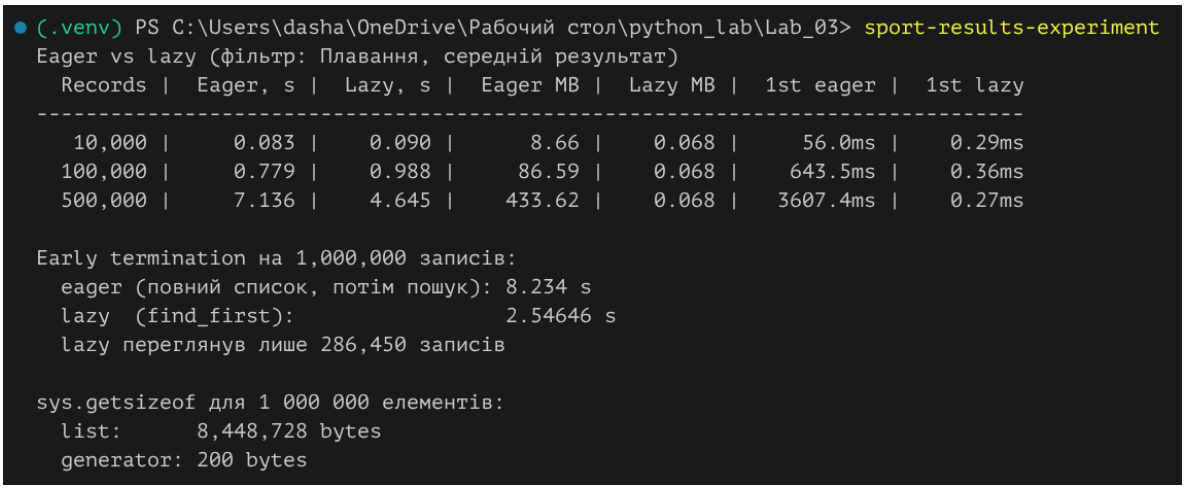


Рисунок 6 — Результати експерименту eager vs lazy

Таблиця elapsed time

Records	Eager time, s	Lazy time, s	Time to first: eager	Time to first: lazy
10 000	0.083	0.090	56.0ms	0.29ms
100 000	0.779	0.988	643.5ms	0.36ms
500 000	7.136	4.645	3607.4ms	0.27ms

Таблиця peak memory

Records	Eager peak memory, MB	Lazy peak memory, MB
10 000	8.66	0.068
100 000	86.59	0.068
500 000	433.62	0.068

Аналіз результатів

- **Peak memory.** Eager-варіант зростає лінійно з розміром набору (≈ 8,7 MB на 10 000 записів, ≈ 434 MB на 500 000), бо одночасно зберігає кілька повних списків. Lazy-варіант тримає сталі ≈ 0,07 MB незалежно від кількості записів: у пам'яті лише поточний record і буфер файлу.

- **Total elapsed time.** На 10 000 і 100 000 записів час майже однаковий, lazy навіть трохи повільніший через накладні витрати на виклики generators (StageLog, кілька stages). На 500 000 записів lazy швидший, бо eager-варіант витрачає час на виділення пам'яті та збирання сміття для великих списків. Отже, generator не завжди швидший — його головна перевага в пам'яті.
- **Time to first result.** Lazy повертає перший результат за частки мілісекунди для будь-якого розміру, а eager має спочатку прочитати й обробити весь файл (секунди на 500 000 записів).
- **Early termination.** На потоці з 1 000 000 записів lazy find_first зупинився після першого збігу, переглянувши лише частину потоку, і виявився в кілька разів швидшим за eager-варіант, що спочатку будує весь відфільтрований список.
- **list vs generator.** sys.getsizeof для списку з 1 000 000 елементів ≈ 8,4 MB (без самих об'єктів), а generator object займає 200 байт незалежно від кількості елементів.
- **Недоліки lazy evaluation.** Generator можна пройти лише один раз; помилки виникають із затримкою — під час споживання; немає доступу за індексом і len(); операції на кшталт sorted і groupby за видом спорту все одно потребують матеріалізації; налагодження складніше, а відкритий файл утримується, доки generator не завершено.

Результат git status

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Рисунок 7 — Результат git status

Результат git log --oneline

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_03> git log --oneline -12
7d0216d (HEAD -> main, origin/main, origin/HEAD) Improve pipeline stage log output
7570124 Add README for Lab_03
7acca20 Add tests for streaming pipeline
fd7506d Add eager vs lazy experiments
a5451e6 Add lazy pipeline, export and entry point
5d94e00 Add custom iterators, batching and streaming analytics
4f94b69 Add streaming readers, validation and filters
0d2e688 Add Lab_03 structure, models and dataset generator
7bbd83b Add Lab_02 report
22cff9e Add README for Lab_02
e45e8a7 Add tests for data processing
e67eca5 Add application entry point
```

Рисунок 8 — Результат git log --oneline (останні 12 комітів)

Відповіді на контрольні запитання

1. Чому iter(iterator) is iterator?

За протоколом метод __iter__() iterator повертає self, тому виклик iter() для iterator повертає той самий об'єкт; це дозволяє використовувати iterator у for.

2. Чи можна повторно використати exhausted generator?

Ні. Після StopIteration generator вичерпаний і повертає порожню послідовність; потрібно створити новий generator.

3. Що таке generator function?

Функція, що містить yield; її виклик повертає generator object, який лениво виробляє значення.

4. Яку роль виконує yield?

Повертає чергове значення викликачу та призупиняє виконання функції зі збереженням її стану до наступного next().

5. Чим yield відрізняється від return?

return завершує функцію й повертає одне значення, yield повертає значення і лише призупиняє функцію, дозволяючи продовжити виконання.

6. Коли фактично починає виконуватися generator function?

Не під час виклику, а при першому next() (або першій ітерації в for).

7. Що зберігає generator між викликами next()?

Локальні змінні, позицію виконання (на якому yield зупинився) та стек викликів функції.

8. Що таке generator expression?

Вираз у круглих дужках (expr for x in items if cond), що створює generator без окремої функції.

9. Чим generator expression відрізняється від list comprehension?

List comprehension одразу створює весь список у пам'яті, generator expression обчислює елементи по одному на вимогу.

10. Що таке lazy evaluation?

Відкладене обчислення: значення обчислюються лише тоді, коли вони потрібні.

11. Що таке eager evaluation?

Негайне обчислення всіх значень і збереження результату, наприклад у list.

12. Які переваги lazy evaluation?

Мала й стала пам'ять, швидкий перший результат, early termination, робота з нескінченними потоками та файлами, більшими за RAM.

13. Які недоліки lazy evaluation?

Одноразовість generator, відкладені помилки, відсутність індексації й len(), складніше налагодження, накладні витрати на виклики.

14. Що таке streaming?

Послідовна обробка даних частинами (record за record) у міру їх надходження без завантаження всього обсягу.

15. Чому streaming важливий для великих files?

Дозволяє обробити файл, більший за оперативну пам'ять, з постійним використанням пам'яті.

16. Чому read() може бути проблемою для multi-GB file?

read() завантажує весь файл у пам'ять одним рядком, що може вичерпати RAM.

17. Чому file object є iterable?

Він реалізує __iter__/__next__ і при ітерації повертає файл рядок за рядком.

18. Для чого використовується yield from?

Для делегування генерації іншому iterable/generator без явного циклу, зокрема для рекурсивних generators.

19. Що таке itertools?

Стандартний модуль з ефективними lazy-будівельними блоками для роботи з iterators.

20. Для чого itertools.count()?

Генерує нескінченну арифметичну послідовність чисел.

21. Чому count() потрібно обмежувати?

Послідовність нескінченна, тому без islice, takewhile або break цикл чи list() ніколи не завершиться.

22. Для чого islice()?

Для зрізу iterator (перші N, з кроком) без матеріалізації, зокрема для обмеження нескінченних потоків.

23. Для чого chain()?

Об'єднує кілька iterables в один послідовний потік.

24. Як працює takewhile()?

Повертає елементи, поки predicate істинний, і зупиняється на першому хибному.

25. Як працює dropwhile()?

Пропускає елементи, поки predicate істинний, а далі повертає всі решту.

26. Для чого accumulate()?

Повертає накопичувальні результати (суми або іншої бінарної функції).

27. Для чого pairwise()?

Повертає пари сусідніх елементів (a, b), (b, c), ...

28. Як працює groupby()?

Групує послідовні елементи з однаковим ключем і повертає пари (key, group-iterator).

29. Чому перед groupby() часто потрібне sorting?

groupby об'єднує лише сусідні елементи; без сортування за ключем однакові ключі утворять кілька окремих груп.

30. Що робить product()?

Повертає декартовий добуток iterables (усі комбінації елементів).

31. Чим combinations() відрізняється від permutations()?

combinations не враховує порядок елементів, permutations враховує, тому перестановок більше.

32. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість результатів зростає експоненційно або факторіально, тому повний перебір займе нереально багато часу.

33. Що таке batching?

Групування елементів потоку в пакети фіксованого або динамічного розміру.

34. Які переваги batching?

Менше накладних витрат на операції (запис у БД, мережа), контрольована пам'ять, можливість паралельної обробки пакетів.

35. Що таке lazy pipeline?

Ланцюжок generators, де кожен stage лениво споживає результат попереднього.

36. Які stages може містити pipeline?

Source, reading, cleaning, parsing, validation, filtering, transformation, batching, aggregation, sink.

37. Що є source у streaming pipeline?

Джерело даних: файл, БД, мережевий потік, генератор.

38. Що є transformation?

Stage, що змінює або збагачує кожен record (нормалізація імені, округлення результату).

39. Що є sink?

Кінцевий споживач потоку: агрегатор, запис у файл, вивід на екран.

40. Чи гарантує generator менший total execution time?

Ні. Експеримент показав, що на малих наборах lazy навіть трохи повільніший; основна перевага — пам'ять і time to first result.

41. Яка основна memory advantage generator?

Він не зберігає всі елементи одночасно, тому пам'ять стала й не залежить від розміру даних.

42. Що таке time to first result?

Час від запуску обробки до отримання першого готового результату.

43. Чому lazy pipeline добре підтримує early termination?

Записи обробляються по одному, тому після знаходження потрібного решту даних можна взагалі не читати.

44. Чому exception у generator може виникати із затримкою?

Код generator виконується лише під час споживання, тому помилка у даних проявляється на next(), а не при створенні generator.

45. Чому list(generator) може бути проблемою?

Він матеріалізує всі елементи в пам'яті й знищує перевагу lazy-обробки для великих наборів.

46. Що трапиться при list(count())?

Програма не завершиться: list намагатиметься зібрати нескінченну послідовність, доки не вичерпається пам'ять.

47. Як generator може впливати на lifecycle file resource?

Файл, відкритий у with усередині generator, залишається відкритим, доки generator не вичерпано або не закрито (close()/збирання сміття).

48. Як тестувати невеликий generator?

Спожити його через list() і порівняти з очікуваним результатом, перевірити next() та StopIteration.

49. Як тестувати infinite generator?

Обмежити його islice або next() кілька разів і перевірити перші значення.

50. Коли краще використовувати list, а коли generator?

List — коли дані невеликі й потрібні кілька проходів, індексація, len() або сортування; generator — для великих або нескінченних потоків, одного проходу та pipelines.

Висновок

У лабораторній роботі побудовано потоковий конвеєр обробки спортивних результатів: source → reading → cleaning → validation → parsing → filtering → transformation → batching/aggregation → result. Реалізовано власний iterator і iterable, generator functions з yield, yield from (у тому числі рекурсивний), generator expressions, нескінченний генератор з islice та засоби itertools chain, accumulate, pairwise, groupby, count. Конвеєр обробляє 100 000 записів із підрахунком некоректних рядків, фільтрацією, рейтингом, pairwise-аналізом, batching і streaming export. Експеримент показав, що lazy-обробка тримає сталу пам'ять ≈ 0,07 MB проти сотень мегабайтів у eager-варіанті, дає перший результат за частки мілісекунди і підтримує early termination, але не завжди швидша за загальним часом. Код покрито 16 тестами pytest і збережено в Git логічними комітами.